

Sistema inteligente de rutas Metro de Medellín

Actividad 2 - Búsqueda y sistemas basados en reglas
Documento de pruebas

Integrantes

Santiago Duque Mora

Geraldine Ríos

Repositorio: <https://github.com/Dumo04/actividad2-rutas-metro-medellin>

Video: Pendiente: Santiago publicará el video en YouTube y aportará el enlace

Docente: Sandra Bautista

Corporación Universitaria Iberoamericana
Ingeniería de Software - Inteligencia Artificial

27/09/2026

1. Descripción del sistema

El sistema responde a la pregunta *¿cuál es la mejor ruta para ir de la estación A a la estación B?* combinando dos técnicas de IA: (1) una **base de conocimiento** escrita en reglas lógicas (syntaxis tipo Prolog/Datalog) que un **motor de inferencia por encadenamiento hacia adelante** procesa para deducir conexiones, transbordos, estaciones disponibles y alcanzabilidad; y (2) una **búsqueda heurística A*** que recorre únicamente los movimientos que las reglas declaran válidos.

Elemento	Valor
Líneas	10
Estaciones	66
Reglas lógicas	11
Hechos base (declarados)	210
Hechos inferidos por el motor	5000
Iteraciones hasta el punto fijo	26
Estaciones de transbordo inferidas	Acevedo, Cisneros, Hospital, Industriales, Miraflores, Oriente, San Antonio, San Javier, Santo Domingo Savio
Penalización por transbordo	5 minutos

1.1 Reglas de la base de conocimiento

```
estacion(X) :- nombre(X, _G1).
conectado(X, Y, L, T) :- tramo(X, Y, L, T).
conectado(X, Y, L, T) :- tramo(Y, X, L, T).
sirve(L, X) :- conectado(X, _G2, L, _G3).
transbordo(X) :- sirve(L1, X), sirve(L2, X), L1 != L2.
disponible(X) :- estacion(X), not cerrada(X).
movimiento(X, Y, L, T) :- conectado(X, Y, L, T), disponible(X), disponible(Y).
vecina(X, Y) :- movimiento(X, Y, _G4, _G5).
alcanzable(X, Y) :- vecina(X, Y).
alcanzable(X, Z) :- alcanzable(X, Y), vecina(Y, Z).
conecta_lineas(L1, L2) :- sirve(L1, X), sirve(L2, X), L1 != L2.
```

Estado = (estación, línea actual). **Costo** = minutos del tramo + penalización si se cambia de línea. **Heurística** $h(n)$ = distancia en línea recta (fórmula de haversine) hasta el destino ÷ velocidad máxima observada en la red (52.4 km/h). Como ningún tramo supera esa velocidad, $h(n)$ nunca sobreestima el costo restante bajo los costos positivos y la penalización no negativa de este modelo. La comprobación se limita a los escenarios evaluados; no valida parámetros arbitrarios.

2. Casos de prueba funcionales

ID	Caso	Resultado esperado	Resultado obtenido	Estado
P1	La Estrella -> Arví con A*	Ruta encontrada, óptima, igual a Costo Uniforme	70 min, 21 paradas, 2 transbordos (A, K, L)	APROBADA
P2	Niquía -> San Javier	Línea A hasta San Antonio y Línea B	37 min, líneas: linea_a, linea_b	APROBADA
P3	Niquía -> Poblado (misma línea)	0 transbordos	27 min, 0 transbordos	APROBADA
P4	Niquía -> San Javier con San Antonio cerrada	Ruta alterna por Metroplús (Hospital -> Cisneros)	Normal 37 min; con cierre 47 min por Acevedo, Hospital, Cisneros, San Javier	APROBADA
P5	Niquía -> Oriente con San Antonio cerrada	Sin ruta: la regla R6 excluye San Antonio y el tranvía queda aislado	Sin ruta	APROBADA
P6	Parque Berrío -> La Estrella: Voraz vs A*	Voraz no garantiza el óptimo; A* sí	Voraz 43 min (3 transb.); A* 23 min (0 transb.)	APROBADA
P7	Consulta lógica transbordo(X)	9 estaciones inferidas	9 resultados	APROBADA
P8	Consulta alcanzable(niquia, arvi)	true	true	APROBADA
P9	Estación ambigua 'Parque'	Error controlado con sugerencias	Error con 3 sugerencias	APROBADA
P10	Nombres sin tildes ni mayúsculas (itagui -> santo domingo savio)	Se resuelven los nombres	Itagüí -> Santo Domingo Savio	APROBADA

10 de 10 casos aprobados.

3. Evidencia de ejecución (salida real de consola)

P1 - Ruta de La Estrella a Arví, comparando algoritmos

```
$ python main.py --origen "La Estrella" --destino Arví --comparar
=====
Base de conocimiento
=====
Reglas: 11   Hechos base: 210   Hechos inferidos: 5000   Iteraciones: 26
Estaciones: 66   Líneas: 10   Transbordos: 9

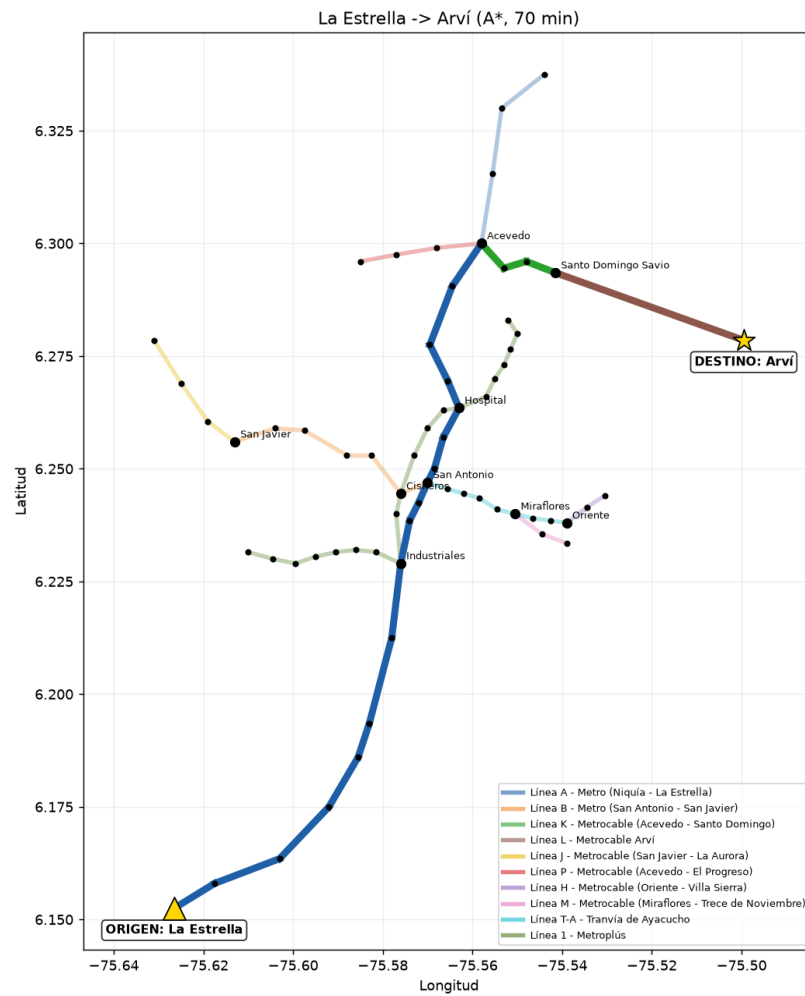
=====
Ruta: La Estrella  ->  Arví   [A*]
=====
1. Tome la Línea A - Metro (Niquia - La Estrella): La Estrella -> Acevedo (17 paradas, 35 min)
   Pasa por: Sabaneta, Itagüí, Envigado, Ayurá, Aguacatala, Poblado, Industriales, Exposiciones, Alpujarra,
             San Antonio, Parque Berrío, Prado, Hospital, Universidad, Caribe, Tricentenario
2. Haga transbordo en Acevedo y tome la Línea K - Metrocable (Acevedo - Santo Domingo): Acevedo -> Santo
   Domingo Savio (3 paradas, 11 min)
   Pasa por: Andalucía, Popular
3. Haga transbordo en Santo Domingo Savio y tome la Línea L - Metrocable Arví: Santo Domingo Savio -> Arví
   (1 paradas, 14 min)
   (directo, sin paradas intermedias)

-----
Tiempo estimado : 70 min (60 en viaje + 2 transbordo(s) x 5 min)
Paradas         : 21
Nodos expandidos: 76   generados: 78   tiempo de cómputo: 0.24 ms

=====
Comparación de estrategias: La Estrella -> Arví
=====


| Algoritmo      | Costo(min) | Paradas | Transb. | Expandidos | ms   |
|----------------|------------|---------|---------|------------|------|
| A*             | 70         | 21      | 2       | 76         | 0.20 |
| Costo uniforme | 70         | 21      | 2       | 77         | 0.12 |
| Amplitud (BFS) | 70         | 21      | 2       | 66         | 0.04 |
| Voraz (greedy) | 70         | 21      | 2       | 44         | 0.12 |


```



P2 - Niquía -> San Javier

```
$ python main.py -o Niquía -d "San Javier"
```

```
=====
Base de conocimiento
=====
```

```
Reglas: 11   Hechos base: 210   Hechos inferidos: 5000   Iteraciones: 26
Estaciones: 66   Líneas: 10   Transbordos: 9
```

```
=====
Ruta: Niquía -> San Javier [A*]
=====
```

1. Tome la Línea A - Metro (Niquía - La Estrella): Niquía -> San Antonio (10 paradas, 20 min)
Pasa por: Bello, Madera, Acevedo, Tricentenario, Caribe, Universidad, Hospital, Prado, Parque Berrío
2. Haga transbordo en San Antonio y tome la Línea B - Metro (San Antonio - San Javier): San Antonio -> San Javier (6 paradas, 12 min)
Pasa por: Cisneros, Suramericana, Estadio, Floresta, Santa Lucía

```
-----
Tiempo estimado : 37 min (32 en viaje + 1 transbordo(s) x 5 min)
Paradas          : 16
Nodos expandidos: 41   generados: 46   tiempo de cómputo: 0.20 ms
```

P4 / P5 - Estación cerrada (San Antonio)

```
$ python main.py -o Niquía -d "San Javier" --cerrar san_antonio -q "alcanzable(niquia, oriente)"
```

```
=====
Base de conocimiento
=====
```

```
Reglas: 11   Hechos base: 211   Hechos inferidos: 3580   Iteraciones: 26
Estaciones: 66   Líneas: 10   Transbordos: 9
Cerradas: San Antonio
```

```
?- alcanzable(niquia, oriente)
false.
```

```
=====
Ruta: Niquía -> San Javier [A*]
=====
```

1. Tome la Línea A - Metro (Niquía - La Estrella): Niquía -> Hospital (7 paradas, 15 min)
Pasa por: Bello, Madera, Acevedo, Tricentenario, Caribe, Universidad
2. Haga transbordo en Hospital y tome la Línea 1 - Metroplús: Hospital -> Cisneros (4 paradas, 12 min)
Pasa por: Ruta N - U de A, Chagualo, Minorista
3. Haga transbordo en Cisneros y tome la Línea B - Metro (San Antonio - San Javier): Cisneros -> San Javier (5 paradas, 10 min)
Pasa por: Suramericana, Estadio, Floresta, Santa Lucía

```
-----
Tiempo estimado : 47 min (37 en viaje + 2 transbordo(s) x 5 min)
Paradas          : 16
Nodos expandidos: 37   generados: 41   tiempo de cómputo: 0.15 ms
```

P6 - Voraz contra A*

```
$ python main.py -o "Parque Berrío" -d "La Estrella" -a greedy --comparar
```

```
=====
Base de conocimiento
=====
```

```
Reglas: 11   Hechos base: 210   Hechos inferidos: 5000   Iteraciones: 26
Estaciones: 66   Líneas: 10   Transbordos: 9
```

```
=====
Ruta: Parque Berrío -> La Estrella [Voraz (greedy)]
=====
```

1. Tome la Línea A - Metro (Niquía - La Estrella): Parque Berrío -> San Antonio (1 paradas, 1 min)
(directo, sin paradas intermedias)
2. Haga transbordo en San Antonio y tome la Línea B - Metro (San Antonio - San Javier): San Antonio -> Cisneros (1 paradas, 2 min)
(directo, sin paradas intermedias)
3. Haga transbordo en Cisneros y tome la Línea 1 - Metroplús: Cisneros -> Industriales (2 paradas, 7 min)
Pasa por: Plaza Mayor
4. Haga transbordo en Industriales y tome la Línea A - Metro (Niquía - La Estrella): Industriales -> La Estrella (7 paradas, 18 min)
Pasa por: Poblado, Aguacatala, Ayurá, Envigado, Itagüí, Sabaneta

```
-----
Tiempo estimado : 43 min (28 en viaje + 3 transbordo(s) x 5 min)
Paradas          : 11
Nodos expandidos: 12   generados: 22   tiempo de cómputo: 0.08 ms
=====
```

Comparación de estrategias: Parque Berrío -> La Estrella

Algoritmo	Costo(min)	Paradas	Transb.	Expandidos	ms
A*	23	11	0	17	0.06
Costo uniforme	23	11	0	53	0.09
Amplitud (BFS)	23	11	0	65	0.04
Voraz (greedy)	43	11	3	12	0.05

P7 / P8 - Consultas a la base de conocimiento

```
$ python main.py -q transbordo(X) -q "conecta_lineas(linea_a, L)" -q "alcanzable(niquia, arvi)"
=====
```

Base de conocimiento

```
=====
Reglas: 11   Hechos base: 210   Hechos inferidos: 5000   Iteraciones: 26
Estaciones: 66   Líneas: 10   Transbordos: 9
```

```
?- transbordo(X)
X = acevedo
X = cisneros
X = hospital
X = industriales
X = miraflores
X = oriente
X = san_antonio
X = san_javier
X = santo_domingo
(9 resultado(s))
```

```
?- conecta_lineas(linea_a, L)
L = linea_b
L = linea_k
L = linea_p
L = metroplus_1
L = tranvia_t
(5 resultado(s))
```

```
?- alcanzable(niquia, arvi)
true.
```

P9 - Estación ambigua

```
$ python main.py -o Parque -d Poblado
=====
```

Base de conocimiento

```
=====
Reglas: 11   Hechos base: 210   Hechos inferidos: 5000   Iteraciones: 26
Estaciones: 66   Líneas: 10   Transbordos: 9
```

```
Error: No encontré la estación 'Parque'. ¿Quiso decir: Parque Berrío, Parque de Aranjuez, Parque de Belén?
```

4. Comparación de estrategias de búsqueda

Se ejecutaron los cuatro algoritmos sobre los mismos pares origen-destino. **A*** y **Costo uniforme** obtienen el mismo menor tiempo en los pares evaluados. A* utiliza la heurística. **Amplitud (BFS)** minimiza el número de paradas, no el tiempo, y **Voraz** es rápido pero no garantiza la ruta óptima.

Origen -> Destino	Algoritmo	Costo (min)	Paradas	Transb.	Nodos expandidos
La Estrella -> Arví	A*	70	21	2	76
La Estrella -> Arví	Costo uniforme	70	21	2	77
La Estrella -> Arví	Amplitud (BFS)	70	21	2	66
La Estrella -> Arví	Voraz (greedy)	70	21	2	44
Niquía -> San Javier	A*	37	16	1	41
Niquía -> San Javier	Costo uniforme	37	16	1	54
Niquía -> San Javier	Amplitud (BFS)	37	16	1	48
Niquía -> San Javier	Voraz (greedy)	37	16	1	17
El Progreso -> Villa Sierra	A*	70	20	3	73
El Progreso -> Villa Sierra	Costo uniforme	70	20	3	78
El Progreso -> Villa Sierra	Amplitud (BFS)	70	20	3	66
El Progreso -> Villa Sierra	Voraz (greedy)	70	20	3	33
Universidad de Medellín -> La Aurora	A*	60	17	2	54
Universidad de Medellín -> La Aurora	Costo uniforme	60	17	2	70
Universidad de Medellín -> La Aurora	Amplitud (BFS)	60	17	2	51
Universidad de Medellín -> La Aurora	Voraz (greedy)	60	17	2	19
Parque Berrío -> La Estrella	A*	23	11	0	17
Parque Berrío -> La Estrella	Costo uniforme	23	11	0	53
Parque Berrío -> La Estrella	Amplitud (BFS)	23	11	0	65
Parque Berrío -> La Estrella	Voraz (greedy)	43	11	3	12
Vallejuelos -> Universidad	A*	37	12	2	23
Vallejuelos -> Universidad	Costo uniforme	37	12	2	30
Vallejuelos -> Universidad	Amplitud (BFS)	47	12	3	29
Vallejuelos -> Universidad	Voraz (greedy)	47	12	3	13
Parque de Aranjuez -> Trece de Noviembre	A*	60	16	3	63
Parque de Aranjuez -> Trece de Noviembre	Costo uniforme	60	16	3	72
Parque de Aranjuez -> Trece de Noviembre	Amplitud (BFS)	60	16	3	54
Parque de Aranjuez -> Trece de Noviembre	Voraz (greedy)	60	16	3	17

Resumen

Algoritmo	Nodos expandidos (total)	Rutas óptimas	Tiempo total (min)
A*	347	7 de 7	357
Costo uniforme	434	7 de 7	357
Amplitud (BFS)	379	6 de 7	367
Voraz (greedy)	155	5 de 7	387

5. Pruebas unitarias automatizadas

Comando: `python -m unittest discover -s tests -v`. Resultado: **27 pruebas ejecutadas, 0 fallos, 0 errores.**

```
test_comparacion (test_logica.TestInferencia.test_comparacion) ... ok
test_negacion_circular (test_logica.TestInferencia.test_negacion_circular) ... ok
test_negacion_estratificada (test_logica.TestInferencia.test_negacion_estratificada) ... ok
test_recursion_cierre_transitivo (test_logica.TestInferencia.test_recursion_cierre_transitivo) ... ok
test_regla_insegura (test_logica.TestInferencia.test_regla_insegura) ... ok
test_regla_simple (test_logica.TestInferencia.test_regla_simple) ... ok
test_error_sintaxis (test_logica.TestParser.test_error_sintaxis) ... ok
test_hechos_y_reglas (test_logica.TestParser.test_hechos_y_reglas) ... ok
test_numeros_cadenas_comentarios (test_logica.TestParser.test_numeros_cadenas_comentarios) ... ok
test_estaciones_de_transbordo_inferidas
  (test_rutas.TestBaseConocimiento.test_estaciones_de_transbordo_inferidas) ... ok
test_lineas_de_san_antonio (test_rutas.TestBaseConocimiento.test_lineas_de_san_antonio) ... ok
test_red_conexa (test_rutas.TestBaseConocimiento.test_red_conexa) ... ok
test_simetria_de_tramos (test_rutas.TestBaseConocimiento.test_simetria_de_tramos) ... ok
test_a_estrella_es_optimo (test_rutas.TestBusqueda.test_a_estrella_es_optimo) ... ok
test_bfs_minimiza_paradas (test_rutas.TestBusqueda.test_bfs_minimiza_paradas) ... ok
test_heuristica_admisible (test_rutas.TestBusqueda.test_heuristica_admisible) ... ok
test_misma_linea_sin_transbordo (test_rutas.TestBusqueda.test_misma_linea_sin_transbordo) ... ok
test_mismo_origen_destino (test_rutas.TestBusqueda.test_mismo_origen_destino) ... ok
test_ruta_continua (test_rutas.TestBusqueda.test_ruta_continua) ... ok
test_ruta_la_estrella_arvi (test_rutas.TestBusqueda.test_ruta_la_estrella_arvi) ... ok
test_voraz_no_garantiza_optimo (test_rutas.TestBusqueda.test_voraz_no_garantiza_optimo) ... ok
test_cierre_con_ruta_alterna (test_rutas.TestEscenarios.test_cierre_con_ruta_alterna) ... ok
test_estacion_ambigua (test_rutas.TestEscenarios.test_estacion_ambigua) ... ok
test_estacion_cerrada_aisla_tranvia (test_rutas.TestEscenarios.test_estacion_cerrada_aisla_tranvia) ... ok
test_estacion_inexistente (test_rutas.TestEscenarios.test_estacion_inexistente) ... ok
test_nombres_flexibles (test_rutas.TestEscenarios.test_nombres_flexibles) ... ok
test_penalizacion_cambia_ruta (test_rutas.TestEscenarios.test_penalizacion_cambia_ruta) ... ok
```

Ran 27 tests in 0.189s

OK

6. Conclusiones

- Separar el conocimiento (archivo .kb) del código permite modificar la red, los tiempos o cerrar estaciones sin reprogramar: el motor de inferencia deduce de nuevo los movimientos válidos.
- Las reglas recursivas (alcanzable) y con negación (disponible) muestran cómo un sistema basado en reglas puede razonar sobre conectividad y restricciones antes de ejecutar la búsqueda.
- En los siete pares evaluados, A* obtuvo el mismo costo óptimo que Costo Uniforme, expandiendo igual o menos nodos.
- BFS y la búsqueda voraz son útiles como comparación: la primera minimiza paradas y la segunda es rápida, pero ninguna garantiza el menor tiempo total.
- Limitación: es un modelo simplificado; los tiempos y coordenadas son aproximados y no se modelan frecuencias ni horarios reales, ni la Línea 2 de Metroplús o las rutas alimentadoras.